Codeforces Round #789 Chinese Tutorial

1678A - Tokitsukaze and All Zero Sequence

注意到当序列中存在 0 时，每次操作选 0 和非 0 的任意数是最优的。

因此当序列中存在 0 时，用cnt表示0的个数,答案为 $n-cnt$。

否则，当序列中不存在 0 时，分两种情况：

1.当序列中存在两个相同的数字时，可以用 1 次操作转化成序列中存在 0 的情况，可以发现这时候答案一定为 n。

2.当序列中所有数字都不相等时，可以用 2 次操作转化成序列中存在 0 的情况，可以发现这时候答案一定为 $n+1$

1678B1 - Tokitsukaze and Good 01-String (easy version)

1678B2 - Tokitsukaze and Good 01-String (hard version)

1.贪心

显然，操作方式是对于每一对相邻且不等的字符，把它们都变成0或者都变成1。换言之，将字符串分成许多相邻的长度为2的二元组，若二元组为"01"或"10"则需要1的操作次数，否则不需要操作次数。

如果希望"连续段"的数量最小，一个简单的贪心方式是：对于每个"需要操作的二元组"（即01或者10），将其变成上一个或者下一个"不需要操作的二元组"（即11或00）的形式。例如："0010"变成"0000"，"1101"变成"1111"。这样这种"需要操作的二元组"对最终的连续段数量是没有任何贡献的。我们只需要统计有多少*连续的*"11"二元组和"00"二元组即可。

特殊的，如果所有二元组均为"01"或"10"，最终对最终连续段的数量总共有1的贡献。

2.dp

显然，操作方式是对于每一对相邻且不等的字符，把它们都变成0或者都变成1。换言之，将字符串分成许多相邻的长度为2的二元组。

考虑从前向后的递推，可以发现对于前面已经维护好的前缀一定以"00"或"11"结尾，所以需要将每一个划分出来的二元组转换成"00"或"11"，通过枚举当前二元组变化成"00"，"11"的代价，在DP的过程中维护改变次数最少的情况下，尽可能的使得当前串与前缀子段的结尾一致。

$dp[n][2]$设第一维为维护好前缀的长度，第二维为前缀的后缀状态。

则有DP转移方程为：

$$dp[i][0]=min(dp[i-2][0]+\{c_0,0\},dp[i-2][1]+\{c_0,1\})$$

$$dp[i][1]=min(dp[i-2][0]+\{c_1,1\},dp[i-2][1]+\{c_1,0\})$$

其中c_k表示为当前二元组转换为"kk"的代价。

1677A - Tokitsukaze and Strange Inequality

我们可以分两步计算答案。

第一步，对于每一个 b，设 f_b 表示区间 $[b+1,n]$ 中小于$p_b > p_d$的p_d的个数。我们可以 $O(n^2)$ 计算出 f。

第二步，计算答案。首先枚举 c 从 1 到 n，再枚举 a 从 1 到 $c-1$。当 $p_a < p_c$ 时，我们可以把区间 $[a+1,c-1]$ 中的 f_b 加进答案。在枚举 a 之前，我们可以先计算 f 的前缀和，这样就能 $\mathcal{O}(1)$ 将区间中的 f_b 加进答案，这一步的时间复杂度是 $\mathcal{O}(n^2)$。但这样会把 d 在区间 $[a+1,c-1]$ 中的结果也加进答案，因为要求 $c<d$，所以这是不合法的。所以我们需要在枚举 c 的同时维护 f：枚举 b 从 1 到 $c-1$，如果 $p_b > p_c$，则 f_b 减去 1。这里的 p_c 实际上看作 p_d，也就是减去 c 与 d 相等的情况，这样就能把不合法的情况减掉。这一步的时间复杂度也是 $\mathcal{O}(n^2)$。

所以总时间复杂度 $\mathcal{O}(n^2)$。

顺便一提，使用 树状数组 或者 线段树 也可以通过，时间复杂度是$\mathcal{O}(n^2 log\ n)$

1677B - Tokitsukaze and Meeting

显然我们可以分开计算行和列的答案

对于列的答案，我们发现由于总共只有$n \cdot m$个学生入场，不会有学生离场，且每次有新学生入场后，所有的列都会循环右移一列，所以答案不会变少。所以每次如果入场的是认真的学生，如果他入场时所在那一列没有认真的学生则列的答案加1，即假如第i个学生是认真的学生，如果之前所有的j, $0 < j < i$,且 $j\%m = i\%m$，都是淘气的学生则列的答案加一。

对于行的答案，我们可以从$i-m$的答案转移过来，相当于$i-m$的答案在上面添加了新的一行，假设上一个认真的学生是第j个，如果$i-j<m$则答案加1，否则答案和第$i-m$个学生入场时相同。

1677C - Tokitsukaze and Two Colorful Tapes

首先暴力找环，取出所有环之后按照：极大值，极小值，极大值，极小值...的顺序填充每一个环。

注意碰到奇环的时候最后一个要空出来填中间值。

举个例子，比如三元环中，如果前两个填1和9，发现中间填$2 \sim 8$的任何一个数字，最终该三元环对于答案提供的贡献不变，所以这种情况就先留出来，最后剩下什么数字就填什么数字。同理，自环，五元环等奇数尺寸的环都由于相同的原因导致最后一个数字不提供任何贡献，最后安排。如此你就可以直接构造出一种$score$最大的解。

我们注意到，如果把每一个环上已经填好的数字取出放到一个新的环状数组h中，实际上产生贡献的数字仅在数组的"山峰"和"山谷"点，我们定义"山峰"为$h_i > h_{i-1}$并且$h_i > h_{i+1}$，"山谷"为$h_i < h_{i-1}$并且$h_i < h_{i+1}$，则显然，每一个"山峰"将提供$2 \cdot h_i$的贡献，每一个"山谷"将提供$-2 \cdot h_i$的贡献。对于非"山峰"和"山谷"的点，它们不提供任何贡献。 为了使score最大，我们令排列中较大的数字成为"山峰"，较小的数字成为"山谷"。对于长度大小为CircleSize的环有$\lfloor \frac{CircleSize}{2} \rfloor$个"山峰"以及"山谷"。

那么最终答案的表达式可以由等差数列求和公式推导得出：令$c = \sum \lfloor \frac{CircleSize}{2} \rfloor$，$N$表示排列的大小，$score = 2c \cdot (N-c)$。

1677D - Tokitsukaze and Permutations

考虑找出序列 v 与排列 p 的关系，发现排列 p 与序列 c 存在双射关系，简单来讲就是序列 c 只会唯一对应一个排列 p，证明：

令 $S = \{1, 2, \ldots, n\}$，发现最后一个数是可以通过 c_n 确定的，因为该位置后面没有数了。可以推出 p_n 为 S 中第 $(c_n + 1)$ 大的数，然后将 S 中 p_n 这个数删除。发现这样子做又可以确定 p_{n-1} 了，于是重复上述过程就可以还原出整个排列。

于是可以考虑对 v 计数，发现进行一次冒泡排序会先使 $v_i = \max(v_i - 1, 0)$，然后 v 整体左移，v_1 直接被覆盖，v_n 置为 0。（因为每一个 v_i 有值的位置前面都有比当前位置大的数，那么该数就一定会被交换，向前移动，v_i 减去 1）举个例子，当前 $v = [0, 0, 2, 1, 1]$，则一次冒泡排序后 $v = [0, 1, 0, 0, 0]$。为了避免混淆，下文的 V 数组为输入中的 v。知道如上结论后就很好计数了，对于位置 $i (i \leq k)$，可以发现 k 次冒泡排序后直接被覆盖了，于是直接将答案乘上 $\prod_{i=1}^{k} i$。对于位置 $i (k < i \leq n)$，如果 $V_{i-k} \neq -1$， 则 v_i 显然唯一确定；如果 $V_{i-k} = -1$，则 v_i 会有 i 种可能，将答案乘上 i；如果 $V_{i-k} = 0$，则 $v_i - k \leq 0$，将答案乘上 $k + 1$。注意对于位置 $i (i \geq n - k + 1)$，应该满足 V_i等于0 或 -1，否则答案一定为 0。

时间复杂度 $\mathcal{O}(n)$。

1677E - Tokitsukaze and Beautiful Subsegments

离线，按升序将查询$[l, r]$按r排序。

将R从1移动到n，并在$r = R$时回答查询$[l, r]$。

使用线段树来解决这个问题，线段树中的每个叶节点$[ul, ul]$维护有多少区间$[ul, R]$是美丽的，然后回答查询就是求和。

同时维护一个单调栈来处理最大值。当R右移一步时，单调栈会弹掉一部分并新增一个。

让我们考虑一下单调栈中的一个数字x，这意味着区间$[lp, R]$ ($l1 \leq lp \leq l2$)的最大值是x。如果在$[l1, l2]$中存在一个满足$a_{ll} * a_R = x$的ll，那么在x被弹出之前，$[l, rr]$区间是美丽的($l \in [l1, l2], l \leq ll, R \leq rr$)。

因此，我们可以假设x在单调栈不会被弹出，这样就相当于线段树上做差分。

使用当前的R作为时间戳，线段树维护一个$a * R + b$的一次函数的形式即可。

所以当R右移时，暴力枚举a_R的倍数更新没有被弹掉的区间，暴力枚举a_R的因子更新新增的区间。

复杂度：$\mathcal{O}(nlognlogn + qlogn)$

1677F - Tokitsukaze and Gems

先来考虑如何计算一条合法路径的贡献，可以发现一条合法路径对答案的贡献相当于下列式子，其中 b_i 表示该路径上第 i 种宝石的个数。

$$\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}(\sum_{j=1}^{n}(P^{i_j}i_j^k)\times\sum_{j=1}^{n}[i_j\neq 0])$$

这个式子看上去似乎无法计算，但观察到后面是链式的结构，考虑 dp，尝试能不能通过只考虑 $1\sim i$ 种宝石的答案推到 $1\sim(i+1)$ 种宝石的答案。考虑维护：

$dp_{n,1}=\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}(\sum_{j=1}^{n}(P^{i_j}i_j^k)\times\sum_{j=1}^{n}[i_j\neq 0])$

$dp_{n,2}=\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}\sum_{j=1}^{n}(P^{i_j}i_j^k)$

$dp_{n,3}=\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}\sum_{j=1}^{n}[i_j\neq 0]$

$dp_{n,4}=\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}1$

观察到如果枚举第 $n+1$ 种宝石选了多少个，这个东西实际上是可以转移的，举个例子。

$dp_{n+1,1}=\sum_{p=0}^{b_{n+1}}\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}((\sum_{j=1}^{n}P^{i_j}i_j^k+P^pp^k)\times(\sum_{j=1}^{n}[i_j\neq 0])+[p\neq 0])$

然后展开。

$A=\sum_{p=0}^{b_{n+1}}\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}(\sum_{j=1}^{n}P^{i_j}i_j^k\times\sum_{j=1}^{n}[i_j\neq 0])=(1+b_{n+1})dp_{n,1}$

$B=\sum_{p=0}^{b_{n+1}}\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}\sum_{j=1}^{n}P^{i_j}i_j^k[p\neq 0]=b_{n+1}dp_{n,2}$

$C=\sum_{p=0}^{b_{n+1}}\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}P^pp^k\times\sum_{j=1}^{n}[i_j\neq 0]=dp_{n,3}\times\sum_{p=0}^{b_{n+1}}P^pp^k=dp_{n,3}\times\sum_{p=1}^{b_{n+1}}P^pp^k$

$D=\sum_{p=0}^{b_{n+1}}\sum_{i_1=0}^{b_1}\sum_{i_2=0}^{b_2}\cdots\sum_{i_n=0}^{b_n}(P^pp^k)\times[p\neq 0]=dp_{n,4}\times\sum_{p=1}^{b_{n+1}}P^pp^k$

现在可以转移了，主要的问题在于如何快速算出 $\sum_{i=1}^{n}i^kx^i$，保证 $x>1$。

设 $h_{k,n}$ 表示 $\sum_{i=1}^{n}i^kx^i$。

根据等比数列求和，有 $h_{0,n}=\dfrac{x^{n+1}-x}{x-1}$。

$$\begin{aligned}xh_{k,n}-h_{k,n}&=\sum_{i=1}^{n}x^{i+1}i^k-\sum_{i=1}^{n}x^ii^k\\&=x^{n+1}n^k+\sum_{i=1}^{n}x^i((i-1)^k-i^k)\\&=x^{n+1}n^k+\sum_{i=1}^{n}x^i([\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}i^j]-i^k)\\&=x^{n+1}n^k+\sum_{i=0}^{k-1}(-1)^{k-j}\binom{k}{j}h_{i,n}\end{aligned}$$

所以最后得出

$$h_{k,n}=\frac{x^{n+1}n^k+\sum_{i=0}^{k-1}(-1)^{k-j}\binom{k}{j}h_{i,n}}{x-1}$$

根据这个递推式得出

$$h_{0,n}=x^{n+1}(\frac{1}{x-1})-x\times(\frac{1}{x-1})$$

$$h_{1,n}=x^{n+1}(\frac{n}{x-1}-\frac{1}{(x-1)^2})-x(-\frac{1}{(x-1)^2})$$

观察可知当 k 固定时存在一个关于 n 的 k 次多项式 $g_k(x)$，使 $h_{k,n}=x^{n+1}g_k(n)-xg_k(0)$ 成立，可以通过数学归纳法证明该结论。

考虑如何用插值得到这个多项式，只需要求出 $g_k(0)\sim g_k(k)$ 即可。

考虑差分，$h_{k,n}-h_{k,n-1}=x^{n+1}g_k(n)-x^ng_k(n-1)=n^kx^n$。

所以可以得出 $xg_k(n)-g_k(n-1)=n^k$。

设 $g_k(0)=C$，于是 $g_k(i)$ 可以表示为 k_iC+b_i 的形式。

但是还是没法解出 C，有一个结论：次数为 n 的多项式 $f(x)$ 的 第$(n+1)$ 次差分为 0。

即 $\sum_{i=0}^{n+1}(-1)^{n+1-i}\binom{n+1}{i}f(i)=0$。

于是就可以解出 $g_k(0)$ 了，然后使用多项式快速插值，再使用多项式多点求值得到需要的每一处点值即可。

发现该 dp 转移可以使用矩阵代替，于是相当于每个位置上有一个矩阵，求所有区间的矩阵乘积之和。这个问题很好解决，你可以用你喜欢的方式去做。

复杂度 $\mathcal{O}(k\log^2 k+n)$。